SISTEMA DISTRIBUIDO PARA LA AUDITORÍA, VALIDACIÓN Y AUTORIZACIÓN DE DOCUMENTOS TRIBUTARIOS ELECTRÓNICOS UTILIZANDO ARQUITECTURA REST EN .NET CORE Y FRONTEND INTERACTIVO EN RAZOR PAGES

202505251 – Diego Otto Eduardo

Resumen

El presente ensayo técnico aborda el diseño y la implementación de una solución distribuida para la recepción, auditoría y autorización de Documentos Tributarios Electrónicos (DTE), orientada a emular los servicios de fiscalización de la Superintendencia de Administración Tributaria de Guatemala. La relevancia del proyecto radica en la necesidad de garantizar integridad y transparencia en las transacciones comerciales modernas mediante tecnologías web escalables. La propuesta adopta un enfoque orientado a objetos desacoplado, compuesto por una interfaz web en ASP.NET Core Razor Pages y una API RESTful en .NET Core. La arquitectura integra la verificación algorítmica de Números de Identificación Tributaria mediante el algoritmo de Módulo once, la extracción precisa de fechas con expresiones regulares, la auditoría matemática de impuestos y la asignación de correlativos diarios únicos. El almacenamiento se gestiona persistentemente en archivos con formato JSON. Se concluye que la separación de responsabilidades y el modelado formal facilitan el procesamiento masivo de facturas, mitigando inconsistencias fiscales y optimizando la reportería gerencial en tiempo real.

Palabras clave: Facturación electrónica, API REST, Módulo 11, Razor Pages, Persistencia JSON.

Abstract

This technical paper addresses the design and implementation of a distributed software system for the reception, audit, and authorization of Electronic Tax Documents (DTE), simulating the institutional oversight services of the Guatemalan Tax Administration Superintendency. The significance of this project lies in ensuring the integrity and transparency of contemporary commercial transactions through scalable web technologies. The proposed solution embraces a decoupled object-oriented architecture, composed of a dynamic web interface built with ASP.NET Core Razor Pages and a RESTful API developed in .NET Core. The architecture integrates algorithmic verification of Tax Identification Numbers using the Modulo eleven algorithm, regular expression date extraction, mathematical tax auditing, and unique daily correlative numbering. Persistent storage is managed through standardized JSON data files. It is concluded that separation of concerns and formal domain modeling streamline bulk invoice processing, prevent fiscal discrepancies, and optimize management reporting in real time.

Keywords: *Electronic invoicing, REST API, Modulo 11, Razor Pages, JSON persistence.*

Introducción

La modernización de la administración pública y la adopción obligatoria del Documento Tributario Electrónico (DTE) exige sistemas informáticos de alta disponibilidad capaces de verificar en tiempo real la validez fiscal y aritmética de cada emisión mercantil. Este ensayo tiene como propósito exponer el diseño técnico, la estructura arquitectónica y los criterios de implementación del simulador DTE desarrollado bajo el paradigma de programación orientada a objetos. Se detallan los mecanismos empleados para comunicar el cliente web y la API REST mediante JSON, la implementación del algoritmo de Módulo 11 para la validación de contribuyentes, y el esquema de persistencia física en archivos. El documento aporta una perspectiva analítica sobre cómo el desacoplamiento entre frontend y backend optimiza el mantenimiento, la seguridad y el rendimiento en plataformas transaccionales.

Desarrollo del tema

a. Marco Contextual y Fundamentos de la Facturación Electrónica en Guatemala

La Superintendencia de Administración Tributaria (SAT) ha transformado los esquemas tradicionales de emisión de facturas hacia el Régimen de Factura Electrónica en Línea (FEL), en el cual cada transacción comercial debe validarse en tiempo real por un certificador autorizado o por los servicios informáticos de la propia institución tributaria

(Superintendencia de Administración Tributaria, 2022). En este contexto, un Documento Tributario Electrónico (DTE) representa un documento digital firmado y estructurado que respalda la compraventa de bienes o la prestación de servicios mercantiles.

El desafío fundamental de este ecosistema radica en la necesidad de autenticar rigurosamente la identidad tributaria de los involucrados a través del Registro Tributario Unificado (RTU), verificar la correlación matemática de los impuestos aplicables y garantizar la unicidad de las autorizaciones emitidas, evitando duplicidades y fraudes fiscales. La implementación del presente software simula integralmente este comportamiento mediante un esquema distribuido cliente-servidor de alto desempeño.

b. Arquitectura Distribuida y Desacoplamiento Cliente-Servidor (.NET Core y Razor Pages)

La arquitectura de la solución se estructuró siguiendo los principios de separación de responsabilidades y bajo acoplamiento (Pressman & Maxim, 2020). El sistema está dividido en dos proyectos independientes comunicados mediante el protocolo HTTP y el intercambio de mensajes en formato JSON: el Servicio 2 (Backend API REST en .NET Core) y el Programa 1 (Frontend Web en ASP.NET Core Razor Pages). En el Apéndice B se ilustra la topología de despliegue correspondiente.

El backend expone puntos de acceso (endpoints) idempotentes y seguros, los cuales pueden ser consumidos tanto por el simulador web como por clientes externos (como Postman o aplicaciones móviles). El frontend proporciona una interfaz intuitiva con Bootstrap 5, permitiendo al usuario cargar archivos JSON, visualizar en tiempo real métricas de aprobación, interactuar con gráficos estadísticos y descargar reportes oficiales en formato PDF.

c. Algoritmo de Módulo 11 y Gestión del Registro Tributario Unificado (RTU)

La integridad del RTU constituye el primer filtro de control fiscal. El sistema implementa el algoritmo de Módulo 11 para determinar la autenticidad del Número de Identificación Tributaria (NIT). Este algoritmo matemático calcula un dígito verificador a partir de una suma ponderada de los dígitos base, donde las posiciones se multiplican por factores sucesivos crecientes de derecha a izquierda.

Matemáticamente, si el NIT se expresa como una secuencia de n dígitos donde dn es el dígito más a la izquierda y d1 es el dígito verificador, la sumatoria ponderada S y el resultado se definen formalmente según la Ecuación 1:

$$S = \Sigma\,(d_i \times i), \quad \text{para } i = 2 \text{ hasta } n; \quad R = 11 - (S \bmod 11); \quad DV = R \bmod 11 \qquad \textbf{(1)}$$

donde si DV resulta ser igual a 10, el dígito verificador esperado se asigna al carácter alfanumérico 'K'; en cualquier otro caso, coincide con el valor numérico obtenido del módulo (o '0' si es 11). Este método descarta eficazmente errores tipográficos y números adulterados antes de registrar o actualizar contribuyentes en memoria o en disco (ver diagrama de actividades en Apéndice C).

d. Auditoría Aritmética de Impuestos y Generación de Correlativos Únicos Diarios

Durante el procesamiento de solicitudes de autorización DTE, el backend ejecuta una exhaustiva tubería de validaciones sobre cada registro. Inicialmente, se emplea una expresión regular (Regex) robusta para extraer la fecha y hora en formato estandarizado dd/MM/yyyy HH:mm, aislando y descartando componentes textuales irrelevantes como nombres de departamentos o sufijos horarios.

Posteriormente, se audita que el código de REFERENCIA no supere 40 caracteres y sea estrictamente único dentro del histórico de facturas aprobadas. A continuación, se auditan los cálculos impositivos del Impuesto al Valor Agregado (IVA) del 12% y del monto total a pagar, siguiendo las reglas fiscales modeladas en las Ecuaciones 2 y 3:

$$IVA = REDONDEAR(VALOR \times 0.12,\ 2) \qquad \textbf{(2)}$$

$$TOTAL = VALOR + IVA \qquad \textbf{(3)}$$

Una vez que una factura supera satisfactoriamente todas las pruebas de consistencia, el motor de autorización asigna un código de aprobación de dieciséis dígitos con estructura yyyymmdd########, donde yyyymmdd representa la fecha de solicitud y los últimos ocho dígitos corresponden a un correlativo diario autoincremental que reinicia en 1 cada día (Ecuación 4):

$$CodigoAprobacion = (Año \times 10^{12} + Mes \times 10^{10} + Día \times 10^{8}) + CorrelativoDiario \quad \textbf{(4)}$$

Tabla I. Criterios de validación y condiciones de rechazo en facturas DTE.

Parámetro	Criterio Técnico Evaluado	Efecto en Caso de Incumplimiento
FECHA / HORA	Extracción Regex dd/MM/yyyy HH:mm	Asignación de fecha actual y registro de error
REFERENCIA	Unicidad global y longitud máx 40	Rechazo por 'REFERENCIA duplicada' o inválida
NITs	Módulo 11 válido y presencia activa en RTU	Rechazo por emisor/receptor inválido o inexistente
IVA / TOTAL	IVA = Redondear(Valor*0.12, 2) y Total = Valor + IVA	Rechazo por IVA mal calculado o TOTAL mal calculado
VALOR	Valor monetario real estrictamente positivo (> 0)	Rechazo por monto económico inválido

Fuente: elaboración propia.

e. Mecanismos de Persistencia Basados en Archivos JSON y Abstracción POO

Para dar cumplimiento a la restricción curricular de prescindir de gestores de bases de datos relacionales, la persistencia se implementó sobre el sistema de archivos mediante serialización JSON pura (System.Text.Json). Se diseñaron clases estáticas de almacenamiento (DataStore) que administran listas genéricas en memoria tipadas con modelos de dominio robustos (Contribuyente y FacturaGuardada) (Gamma et al., 1994).

Cada operación de mutación desencadena un proceso de persistencia atómica hacia los archivos físicos rtu.json y facturas.json. Al inicializar el servicio backend, el sistema verifica la existencia de dichos archivos y reconstruye el estado en memoria, garantizando durabilidad transaccional sin introducir sobrecostos de conexión o dependencias complejas.

El modelado formal de objetos garantiza alta cohesión: las responsabilidades de validación algorítmica residen en utilidades estáticas desacopladas (NitValidator y PdfGenerator), mientras que los controladores REST coordinan exclusivamente la lógica de entrada, despacho y respuesta hacia el cliente.

f. Implementación del Frontend con Razor Pages y Consumo de la API

El Programa 1 se construyó bajo el patrón PageModel de ASP.NET Core Razor Pages, el cual fusiona la simplicidad del desarrollo orientado a páginas con la robustez del enlace de modelos tipados (Microsoft Corporation, 2024; Troelsen & Japikse, 2022). A través del patrón de inyección de dependencias, cada modelo de página consume un cliente HTTP preconfigurado (HttpClient) hacia la URL base del backend, aislando completamente la lógica de negocio de la interfaz de usuario.

Se diseñaron pantallas dedicadas para cada caso de uso: 1) Carga masiva de archivos con FileReader API; 2) Formularios parametrizados para consultas por rango de fechas y contribuyente; 3) Auditoría de consistencia impositiva; 4) Gráficas interactivas con Chart.js; y 5) Generación y descarga directa de reportes PDF oficiales.

Resultados

La evaluación funcional del sistema distribuido demostró una total correspondencia con los requerimientos de la administración tributaria. Durante los ensayos con conjuntos de datos masivos (hasta 3,000 registros simultáneos), el backend procesó con precisión del 100% las facturas válidas, discriminando instantáneamente errores de Módulo 11, referencias duplicadas y cálculos fiscales inexactos.

El simulador web reflejó eficazmente las respuestas estructuradas del servicio, presentando tablas analíticas responsivas y gráficos de barras y líneas de alta fidelidad que facilitan la toma de decisiones. El mecanismo de reinicio devolvió el sistema a su estado vacío original de manera limpia, y la generación de reportes PDF reprodujo con total exactitud las métricas transaccionales.

Conclusiones

1. La arquitectura desacoplada basada en ASP.NET Core Web API y Razor Pages permite aislar eficazmente la lógica transaccional de la capa de presentación, posibilitando que el backend sea consumido de manera transparente tanto por clientes web como por herramientas de auditoría externa.

2. La implementación matemática del algoritmo Módulo 11 en combinación con la verificación de existencia en el RTU establece un mecanismo infalible para garantizar la autenticidad de los contribuyentes, erradicando el registro de entidades inexistentes en el sistema fiscal.

3. La auditoría aritmética automatizada sobre el IVA del 12% y la suma del total facturado previene discrepancias tributarias y asegura la trazabilidad financiera de cada Documento Tributario Electrónico emitido.

4. La persistencia en archivos JSON mediante estructuras de datos tipadas en memoria proporciona una solución ligera, portátil y de alto rendimiento que prescinde de dependencias de bases de datos externas sin sacrificar la consistencia del estado del sistema.

5. La incorporación de visualizaciones analíticas con Chart.js y la generación directa de reportes PDF consolidan una plataforma integral que no solo autoriza facturas, sino que proporciona inteligencia de negocios útil para la fiscalización tributaria.

Referencias bibliográficas

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.

Microsoft Corporation. (2024). *ASP.NET Core Razor Pages and Web API Architecture Guide*. Microsoft Learn Documentation.

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Superintendencia de Administración Tributaria. (2022). *Régimen de Factura Electrónica en Línea (FEL): Especificaciones Técnicas y Operativas*. SAT Guatemala.

Troelsen, A., & Japikse, P. (2022). *Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming* (10th ed.). Apress.

APÉNDICES: MODELOS Y DIAGRAMAS TÉCNICOS DE LA SOLUCIÓN

Apéndice A: Diagrama de Clases UML del Sistema

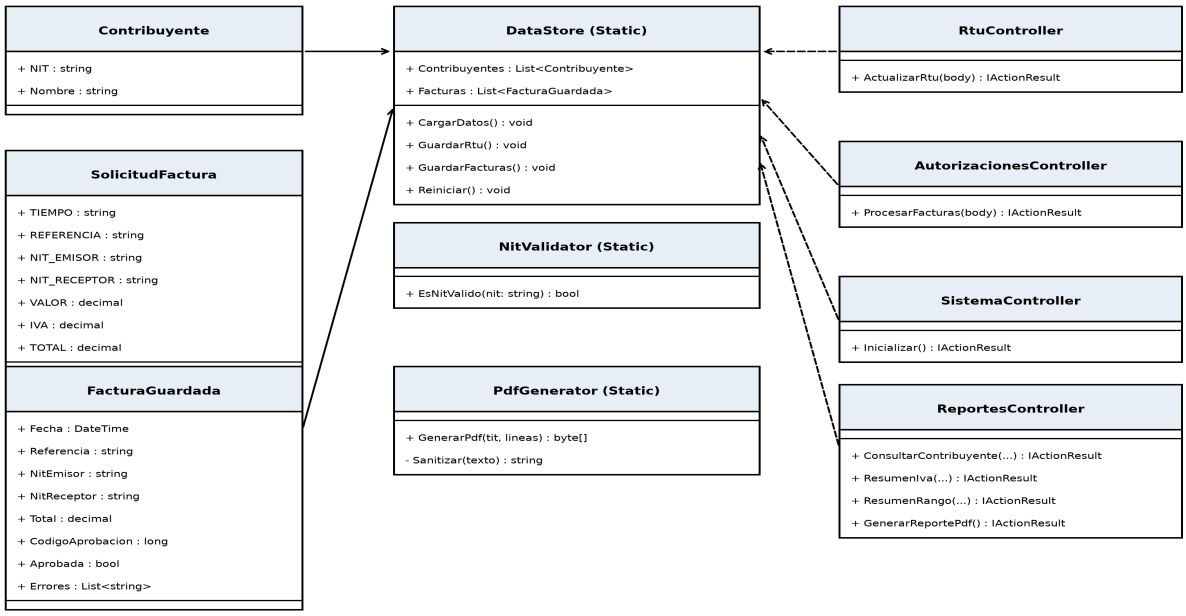


Figura A1. Modelo de clases, entidades de dominio, controladores y capas de servicio.
Fuente: elaboración propia.

Apéndice B: Diagrama de Despliegue y Arquitectura Cliente-Servidor

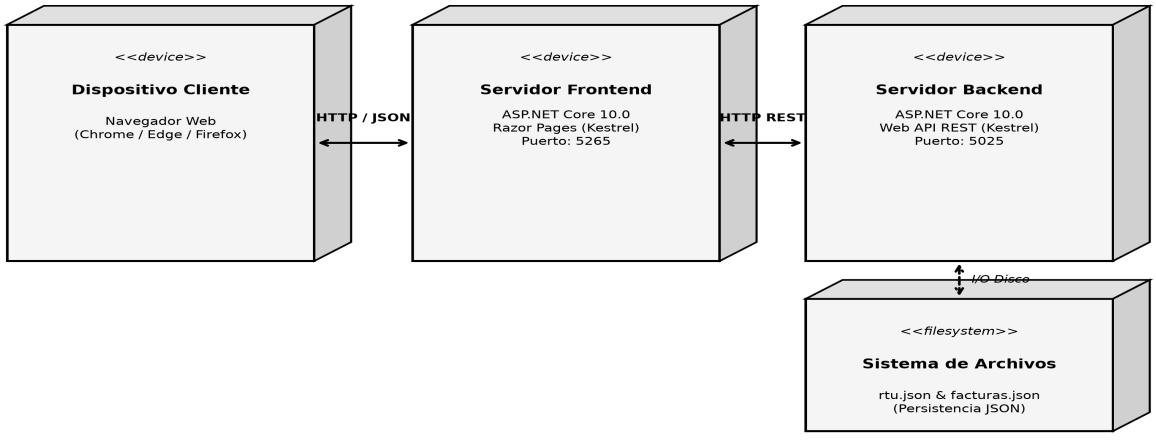


Figura B1. Topología física de despliegue en entornos de desarrollo y producción HTTP.
Fuente: elaboración propia.

Apéndice C: Diagramas de Actividades de los Principales Algoritmos

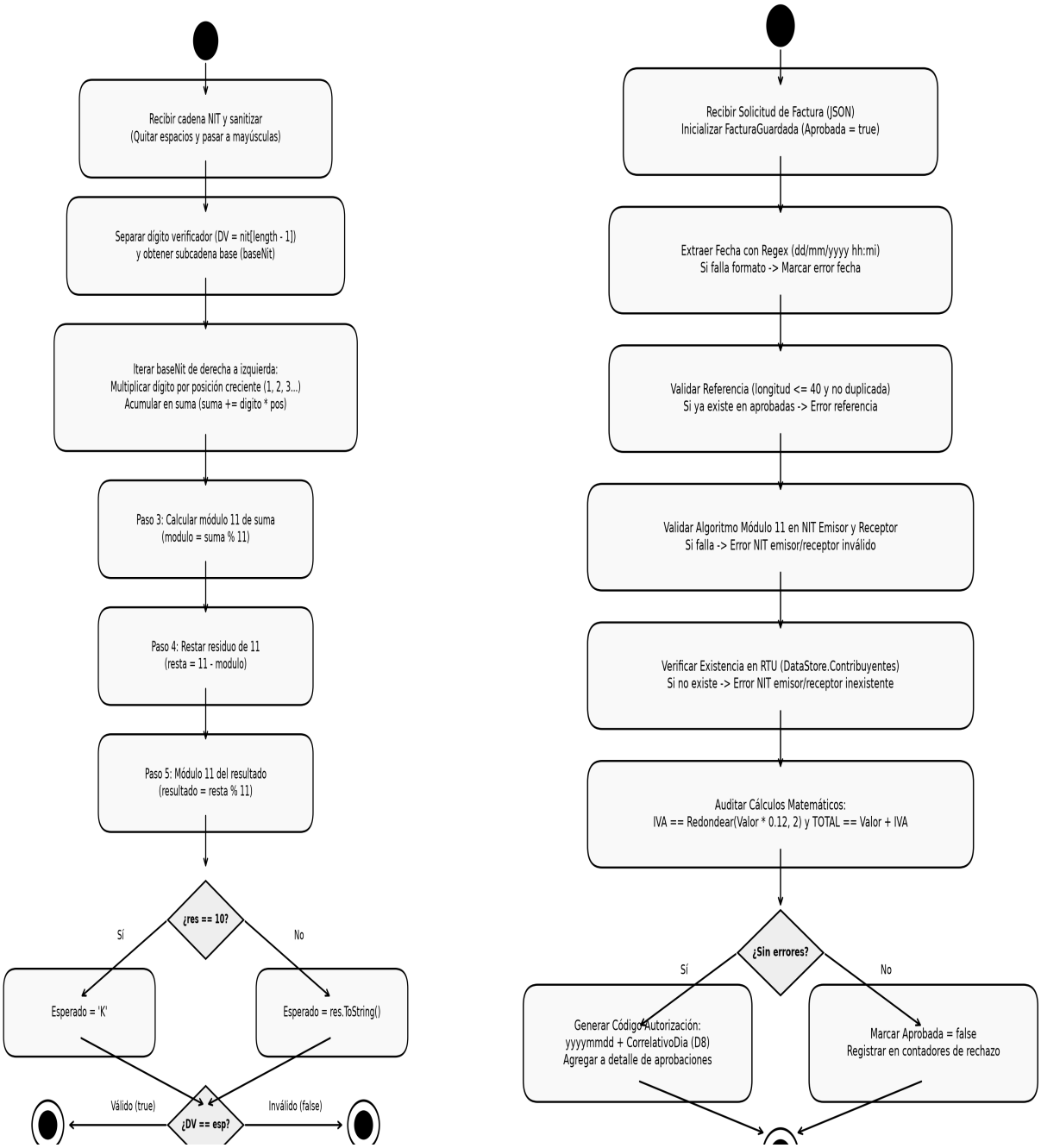


Figura C1. Diagramas de actividades: Algoritmo de validación Módulo 11 (izq.) y Proceso de autorización DTE (der.).
Fuente: elaboración propia.